

	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Servicios				CURSO: 2º
	PROTOCOLO:	Examen	AVAL:	2ª	DATA:	19/02/26
	Nombre y Apellidos:					

En las lejanas y frías tierras del norte, pero muy muy al norte, los helados atómicos vivían en una época en que se enfrentaba a terribles problemas de stress. Y claro, si uno de los helados se ponía nervioso de más, explosionaba. Y dicho estallido no era precisamente leve, si no que más bien dejaría anonadado a Oppenheimer y su cuadrilla. Esta preocupación tenía a DobleVainilla sin dormir, porque veía que su amigo y vecino Spi... estoo, PoloFresa, estaba cada día más ansioso, y nuestro delicioso protagonista se temía lo peor. Así que una mañana se levantó de su congelador, se vistió con su cucurucho y su topping multicolor, y se fue a la casa de su amigo para planificar una lista de ejercicios de relajación. Se les ocurrieron tantos que pensaron en crear una base de datos a la que ellos y otros amigos como Bananasplit o Sorbetelimón pudieran acceder a estos ejercicios de una forma cómoda desde casa.

El problema es que no tenían muy claro como realizar la parte de red y mandaron un mail a los avezados, geniales y magos de los servidores: los alumnos del Colegio Vivas. Les pidieron que, en una primera fase, hiciesen un gestor de usuarios para acceso a una futura base de datos remota.

Así que las especificaciones quedaron de la siguiente manera:

Se debe realizar un proyecto **consola**. El nombre del proyecto será **ApellidosNombre_Serv2aEv**.

Se desea realizar un gestor de usuarios para acceso a una base de datos en red con funcionalidad limitada. A este gestor se pueden conectar clientes que dispongan de cuenta. La estructura será la siguiente:

(0.5p) Habrá una **clase User** con las siguientes propiedades y características:

- **Name:** string que representa el nombre de usuario con set/get estándar.
- **Password:** string que representa la password de usuario con set/get estándar.
- Crea un **constructor** que inicialice ambas propiedades según parámetros.
- Sobreescribe **Equals** de forma que devuelve true si hay coincidencia en Nombre y Password.

Habrá una **clase Server** con al menos los siguientes miembros **no estáticos**:

- **users:** Colección privada de objetos tipo User.
- **(1p)** Método privado **freePort** que devuelve un int: Se le pasa como parámetro un vector de **ushort**. Evalúa por orden los elementos del vector de forma que si encuentra uno que corresponda con un puerto libre lo devuelve. Si ninguno estuviera libre devuelve -1.

- **(1p)** Método público **Init** con las siguientes características:

Llama a `freePort` pasando un vector con los puertos: 135, 49665, 49666, 31416, 5000. Si no hay ninguno libre la función `Init` finaliza.

Inicializa la colección *users* con 5 usuarios.

A continuación inicializa el servidor en el puerto obtenido mediante `freePort`. En pantalla indica el puerto en el que se queda a la espera. Luego se establece un bucle de espera y recepción de clientes estándar de forma que si uno realiza una conexión, se ejecuta en un hilo aparte llamando a la función *ClientThread* descrita a continuación.

- **Método privado ClientThread** de atención al cliente:

(0.5p) Tendrá un parámetro de tipo `Socket`. Inicializa los Streams habituales para poder hacer comunicación por red. Lo primero que se hará es enviarle la palabra "**LOGIN:**" al cliente.

(1p) A continuación se queda esperando a que el cliente mande en una única línea el usuario y la password en el formato: **nombre;password**

Si el usuario no está en la colección *users* o la pass no es válida se termina la conexión. Si has realizado bien la sobreescritura de `Equals` de `User`, recuerda que puedes usar el `Contains` de la colección.

Si es correcto se mantiene conectado y puede ejecutar uno de estos comandos (si es no válido no hace nada pero se mantiene conectado).

Los comandos que puede enviar el cliente son:

- **(0.5p) ul** : (User List) Devuelve la lista de usuarios completa (solo nombres).
- **(1p) adduser nombre pass** : Comando que añade un nuevo usuario a la colección con nombre y pass indicado. No debe haber nombres duplicados. Ni el nombre ni la password pueden tener espacios.
- **(1p) deluser inicio_nombre** : Elimina todos los usuarios de la colección cuyo nombre comience por *inicio_nombre*. Además por cada nombre que se borre se **añade** una entrada en un archivo de logs con el siguiente formato:

[Fecha y hora] – Nombre usuario borrado

Fecha y hora obtenlo mediante `DateTime.Now`.

El archivo se denomina *commands.log* y se guarda en *%userprofile%*. No es necesario tener en cuenta problemas con este archivo.

- **(0.5p) exit** : El cliente se desconecta.

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Servicios				CURSO: 2º
	PROTOCOLO:	Examen	AVAL:	1ª	DATA:	2/3/2020
	Nombre y Apellidos:					

- **(1p) halt** : Cierra el servidor de forma natural (No uses Environment.Exit ni similares para finalizar la aplicación). Si hubiera alguien conectado, debe ser desconectado.

Además:

(1p) El servidor no debe fallar ante **cierres abruptos** de los clientes, es decir, se debe controlar que si se cierra el telnet de golpe, que el servidor no falle y se cierren todos los recursos del cliente adecuadamente.

(1p) El **acceso a recursos comunes** debe estar siempre correctamente controlado si hay varios hilos que puedan acceder al mismo tiempo a dicho recurso.

En el main, que estará en la clase Program, simplemente crea un objeto del tipo Server y ejecuta su función Init.

No se usarán excepciones genéricas pero sí IOException o SocketException sin necesidad de especificar el código de error.

Por si tienes dudas la colección NO se guarda en ningún archivo.

NOTAS:

- Añade tu nombre y apellidos al principio de cada archivo.
- Recuerda: los hilos deben ser robustos ante el acceso a elementos comunes.
- Tanto los nombres de identificadores como los distintos textos usados en el interfaz de usuario se tratarán de expresar en inglés.
- Usa IOException o SocketException según necesites. No uses Exception. No es necesario el uso de códigos de error de SocketException.
- Debes usar tipado de variables, **no** se permite el uso de **dynamic** o de **var** si no es claro el tipo.
- Despreocúpate de las codificaciones de los mensajes (UTF, ASCII,...).
- Acuérdate de los **Flush** o de **AutoFlush**.
- Pregunta al profesor antes de realizar más métodos de los aquí expuestos o cambiar la estructura indicada en la especificación (por defecto no se puede y podría afectar a la nota).

- Algunas sentencias/elementos que pueden ser útiles:
 - El puerto 135 siempre está ocupado en Windows. Puedes ver más mediante el comando `netstat -ao`
 - `Environment.GetEnvironmentVariable()`
 - `Environment.ExpandEnvironmentVariables()`
 - `Environment.NewLine`
 - `NetworkStream`
 - `StreamReader`
 - `StreamWriter`
 - Funciones estáticas de `File`.
 - `Socket s = new Socket(AddressFamily.InterNetwork, SocketType.Stream, ProtocolType.Tcp)`
 - `IPEndPoint`
 - `IOException`
 - `SocketException`
 - Funciones de la clase `Array` en combinación con método `ToArray()` de las colecciones.
 - Funciones de `string`: `IndexOf`, `Substring`, `Split`, `Contains`, etc...